

《俄罗斯方块：技能大作战》程序设计实习大作业报告

所属组别：第 12 组

项目名称：Tetris Skill Battle

技术栈：C++17 / Qt Widgets / CMake / Python

一、程序功能介绍

本项目是一款基于 C++17 与 Qt Widgets 开发的桌面端俄罗斯方块游戏。项目在经典 20×10 棋盘规则的基础上，引入了技能、能量、成长解锁与局前配装机制，将传统方块游戏中的即时操作与战略规划结合起来，形成了“基础玩法稳定、进阶玩法有差异化”的课程大作业成果。

1. 核心玩法与模式设计

- 经典模式保留纯粹的俄罗斯方块体验，不启用技能与能量系统，适合验证基础操作手感与分数成长节奏。
- 无限挑战模式是本项目的核心特色模式。玩家通过消行积累公共能量池，在局内使用数字键 1-4 或默认快捷键 V/E 释放已装备技能，以应对高压堆叠和复杂局面。
- 项目实现了预览方块、幽灵方块、暂停、重开、结算弹窗和排行榜记录等完整的局内流程，使游戏具备较完整的单机游玩闭环。

2. 技能、能量与成长机制

技能系统围绕“资源积累 - 决策释放 - 进度成长”展开。能量池上限固定为 140，单次清除 1 至 4 行分别获得 20、35、50、80 点能量，发生连击时还会追加额外能量奖励，从而鼓励玩家争取高质量消行而不是单纯拖延时间。

技能名称	类型	效果概述	能量消耗	解锁条件
时间凝滞	持续型	显著降低当前下落速度，为复杂堆叠争取调整空间。	35	初始解锁
底部爆破	瞬发型	清除最底部 1 行并触发整体下落。	45	初始解锁
直线清扫	瞬发型	删除当前最危险的一行，适合中期止损。	60	最高分 ≥ 1200
大地震	瞬发型	强制清除底部 2 行并压低地形。	80	最高分 ≥ 3000
失重领域	持续型	短时间内大幅降低重力，适合高压重整局面。	95	最高分 ≥ 5500
净场脉冲	瞬发型	清除底部 4 行，代价高但容错极强。	120	最高分 ≥ 9000

成长机制与历史最高分绑定：初始仅开放 2 个携带槽位和两项基础技能，随着最高分提升逐步开放更高代价但更强力的技能，同时最多可扩展到 4 个携带槽位。该设计既控制了新手学习成本，也让高分玩家拥有更强的策略组合空间。

3. 本地持久化与辅助工具链

- ProgressManager 基于 QSettings 记录最高分、本地 Top 5 排行榜、已解锁技能列表和玩家最近一次配装。
- 项目在 src/data/skills.json 中维护技能的展示元数据，并额外提供 Python 脚本用于导入、导出和调整技能数据。
- 这种 “C++ 核心逻辑 + JSON 配置 + Python 辅助工具” 的组合，使策划调参、演示准备和后期维护更加方便。

二、项目各模块与类设计细节

项目采用 CMake 组织构建，代码总体按 “核心逻辑层、界面视图层、数据配置层、脚本工具层” 拆分，分别对应仓库中的 core、ui、data 和 python 目录。整体结构接近 “逻辑层与表现层分离” 的轻量化 MVC 思路。

模块	代表类 / 文件	职责说明
核心逻辑层	GameEngine / TetrisBoard / Tetromino / RandomBag	维护棋盘状态、方块生成、旋转与碰撞判定、主循环更新以及游戏状态机。
技能系统	SkillManager / SkillBase / SkillFactory / 各技能类	处理能量池、技能实例化、激活生命周期与局内效果叠加。
成长与存档	ProgressManager / skills.json / Python 辅助脚本	记录最高分、排行榜、解锁进度和配装方案，并支持数据维护。
界面视图层	MainWindow / GameBoardWidget / MenuWidget / SkillPanelWidget	负责界面组织、输入采集和状态可视化。
表现特效层	ScreenFlash / ScreenShake / ParticleSystem	对技能释放和消行动作提供闪屏、震屏和粒子反馈，提升交互表现力。

1. 核心逻辑模块 (src/core)

- GameEngine 是项目的主控引擎与状态机，负责开始、暂停、重开、结算、主循环 Tick 更新以及键盘输入分发。
- TetrisBoard 维护包含 2 行缓冲区的 22×10 网格，提供碰撞检测、整行清除、技能清行和整体重力压缩等能力。
- Tetromino 以 4×4 形状矩阵封装 7 种方块，并实现了 SRS 旋转与踢墙判定，是玩法正确性的关键基础。

- RandomBag 采用 7-bag 随机算法构造出块序列，避免极端缺块现象，提高了玩法公平性。
- ScoreCalculator 统一管理分数、等级、总消行数与 Combo，并通过等级影响下落速度，形成稳定的难度曲线。

2. 技能系统架构 (src/core/skills)

技能系统采用“抽象基类 + 工厂注册 + 运行时实例化”的设计。SkillBase 约定了 onActivate、onTick、onDeactivate 等生命周期接口，SkillFactory 通过宏 REGISTER_SKILL 将技能标识映射为可实例化的具体技能类。SkillManager 则统一管理能量消耗、默认槽位、激活状态与技能使用统计。

- 时间凝滞、失重领域等持续型技能通过修改重力倍率影响局内节奏。
- 底部爆破、大地震、净场脉冲等瞬发型技能直接操作棋盘底部行，配合 applyGravity 形成明显的局面重置效果。
- 技能系统与 GameEngine
保持弱耦合：引擎只负责更新与广播，具体技能效果由派生类内部实现，便于后续扩展新玩法。

3. 界面与表现层 (src/ui)

- MainWindow 负责菜单界面、对局界面与结算对话框之间的切换，并驱动 QTimer 构成约 60 FPS 的刷新循环。
- GameBoardWidget 通过 QPainter 绘制棋盘网格、当前方块、幽灵方块和锁定后的落点内容，同时监听键盘事件并映射到 GameAction。
- MenuWidget、EnergyBarWidget、SkillPanelWidget、ScorePanelWidget 和 NextPieceWidget 共同承担菜单、分数、能量、技能槽和预览区的可视化职责。
- ParticleSystem、ScreenFlash 与 ScreenShake 为技能释放和消行动作提供了直观反馈，让课程项目不只“能玩”，而且具备较完整的表现层质感。

4. 数据与工程组织

项目根目录以 CMake 构建脚本为入口，兼容 Qt5 与 Qt6 的 Widgets 开发方式。release 目录提供了已打包的 Windows 可执行版本，便于课程展示与结果验收。技能元数据与辅助脚本被拆分到独立目录，减轻了核心代码与数值配置相互缠绕的问题。

三、小组成员分工情况

1. 吴科宣 (队长, 学号: 2400017810)

- 主导项目整体架构设计与大部分核心代码实现，重点负责 GameEngine、TetrisBoard 与 Tetromino 等关键类。
- 统筹主界面与逻辑层的联调，完成主要交互流程的闭环与整体可玩性打磨。
- 负责成果演示相关内容的整合，包括录屏、展示素材整理与最终成品呈现。

2. 赵炜峰 (组员, 学号: 2500012774)

- 在项目早期完成原型级 Demo 的验证工作, 为碰撞、消行与方块下落等基础机制提供技术可行性依据。
- 参与技能玩法设计、能量消耗规划以及平衡性讨论, 使“技能大作战”与经典方块玩法能够有效结合。
- 支持中期功能扩展与联调测试, 帮助识别对局过程中的规则边界与异常场景。

3. 唐永安 (组员, 学号: 2500013155)

- 负责工程目录整理、CMake 配置维护和辅助脚本组织, 提升项目结构清晰度与可维护性。
- 主导课程文档编写, 将项目中的模块关系、核心思路与实现细节沉淀为规范化报告。
- 承担中后期合并、收尾与打包任务, 确保最终版本能够稳定交付并满足展示要求。

四、项目总结与反思

1. 项目总结

通过本次大作业, 我们完成了一个从规则实现到界面整合、从单机玩法到成长系统扩展的完整桌面游戏项目。项目实践让我们将课堂中的封装、继承、多态、模块化构建和数据驱动思想落实到了可运行的软件中。特别是技能系统的抽象设计、7-bag 随机机制与本地持久化方案, 为后续继续扩展玩法提供了较好的基础。

2. 反思与不足

- 当前项目仍存在一定程度的 UI 与逻辑联动耦合, 例如部分状态刷新依赖界面组件主动监听信号, 后续可以进一步抽象视图模型以简化交互边界。
- 技能对象与表现特效虽然整体可控, 但在复杂场景下仍需要更系统的性能分析与内存检查工具进行验证。
- 排行榜与成长数据目前仅保存在本地, 后续若引入 Qt Network 或服务端接口, 可继续扩展在线排行、云存档甚至局域网对战模式。

总体而言, 本项目已经达成了课程大作业对于面向对象设计、图形界面开发和完整软件交付的综合训练目标, 同时也为我们后续继续迭代更复杂的游戏系统积累了工程经验。